Naive Bayes Classifier

```
import numpy as np

class NaiveBayesClassifier:
    def __init__(self, alpha=1.0):
        self.alpha = alpha
        self.classes = None
        self.n_features = None
        self.feature_values = None
        self.prior = {}
        self.conditional_prob = {}
        self.class_counts = {}

    def fit(self, X, y):
        X = np.asarray(X, dtype=object)
        y = np.asarray(y)
        self.classes = np.unique(y)
        self.n_features = X.shape[1]
        self.feature_values = [
            np.unique(X[:, j])
            for j in range(self.n_features)]

        self.prior = {}
        self.conditional_prob = {}
        self.class_counts = {}

        for cls in self.classes:
            X_cls = X[y == cls]
            n_cls = len(X_cls)
            self.class_counts[cls] = n_cls
            self.prior[cls] = n_cls / len(y)
            self.conditional_prob[cls] = {}

            for j in range(self.n_features):
                self.conditional_prob[cls][j] = {}
                k = len(self.feature_values[j])

                for value in self.feature_values[j]:

                    count = np.sum(X_cls[:, j] == value)
                    prob = (count + self.alpha) / (n_cls + self.alpha
* k)
                    self.conditional_prob[cls][j][value] = prob

        return self

    def predict(self, X):
```

Name - Mahima Kumari
Roll No. - 24
SIC - 24BCSF87
Date - 9 Nov 2026

```
        X = np.asarray(X, dtype=object)
        predictions = []

        for instance in X:
            class_scores = {}
            for cls in self.classes:
                probability = self.prior[cls]

                for j, value in enumerate(instance):

                    probabilities = self.conditional_prob[cls][j]
                    if value in probabilities:
                        conditional = probabilities[value]
                    else:
                        k = len(self.feature_values[j])
                        conditional = self.alpha /
(self.class_counts[cls] + self.alpha * k)

                    probability *= conditional

                class_scores[cls] = probability

            predicted_class = max(class_scores, key=class_scores.get)
            predictions.append(predicted_class)

        return np.array(predictions)

#Play Tennis Dataset
X_train = np.array([
    ["Sunny", "Hot", "High", "Weak"],
    ["Sunny", "Hot", "High", "Strong"],
    ["Overcast", "Hot", "High", "Weak"],
    ["Rain", "Mild", "High", "Weak"],
    ["Rain", "Cool", "Normal", "Weak"],
    ["Rain", "Cool", "Normal", "Strong"],
    ["Overcast", "Cool", "Normal", "Strong"],
    ["Sunny", "Mild", "High", "Weak"],
    ["Sunny", "Cool", "Normal", "Weak"],
    ["Rain", "Mild", "Normal", "Weak"],
    ["Sunny", "Mild", "Normal", "Strong"],
    ["Overcast", "Mild", "High", "Strong"],
    ["Overcast", "Hot", "Normal", "Weak"],
    ["Rain", "Mild", "High", "Strong"]], dtype=object)

y_train = np.array([
    "No", "No", "Yes", "Yes", "Yes", "No", "Yes", "No", "Yes", "Yes",
"Yes", "Yes", "Yes", "No"])

model = NaiveBayesClassifier(alpha=1.0)
model.fit(X_train, y_train)
```

Name - Mahima Kumari
Roll No. - 24
SIC - 24BCSF87
Date - 9 Nov 2026

```python
X_test = np.array([
    ["Sunny", "Cool", "High", "Strong"],
    ["Overcast", "Mild", "Normal", "Weak"],
    ["Rain", "Mild", "High", "Strong"]], dtype=object)

predictions = model.predict(X_test)
print("Predictions on Test Data:")
for instance, prediction in zip(X_test, predictions):
    print(instance, "=>", prediction)

print("\nPredictions on Training Data:")
train_predictions = model.predict(X_train)
for instance, prediction in zip(X_train, train_predictions):
    print(instance, "=>", prediction)

# Accuracy on Training Data
train_predictions = model.predict(X_train)
train_accuracy = np.mean(train_predictions == y_train)
print("\nTraining Accuracy:", round(train_accuracy*100,2),"%")

# For the test set, you need true labels to compare
y_test = np.array(["No", "Yes", "No"])  # ground truth for your X_test
test_accuracy = np.mean(predictions == y_test)
print("Test Accuracy:", test_accuracy)
```

```
Predictions on Test Data:
['Sunny' 'Cool' 'High' 'Strong'] => No
['Overcast' 'Mild' 'Normal' 'Weak'] => Yes
['Rain' 'Mild' 'High' 'Strong'] => No

Predictions on Training Data:
['Sunny' 'Hot' 'High' 'Weak'] => No
['Sunny' 'Hot' 'High' 'Strong'] => No
['Overcast' 'Hot' 'High' 'Weak'] => Yes
['Rain' 'Mild' 'High' 'Weak'] => Yes
['Rain' 'Cool' 'Normal' 'Weak'] => Yes
['Rain' 'Cool' 'Normal' 'Strong'] => Yes
['Overcast' 'Cool' 'Normal' 'Strong'] => Yes
['Sunny' 'Mild' 'High' 'Weak'] => No
['Sunny' 'Cool' 'Normal' 'Weak'] => Yes
['Rain' 'Mild' 'Normal' 'Weak'] => Yes
['Sunny' 'Mild' 'Normal' 'Strong'] => Yes
['Overcast' 'Mild' 'High' 'Strong'] => Yes
['Overcast' 'Hot' 'Normal' 'Weak'] => Yes
['Rain' 'Mild' 'High' 'Strong'] => No

Training Accuracy: 92.86 %
Test Accuracy: 1.0
```

Name - Mahima Kumari
Roll No. - 24
SIC - 24BCSF87
Date - 9 Nov 2026